Introduction to Statistical Learning and Application - Master of Applied Mathematics

Razan MHANNA

STATIFY team, Inria, University Grenoble Alpes
Team 5, Grenoble Institute of Neurosciences GIN

March 5, 2026

Outline

1. Classification
2. Logistic Regression
3. SVM
4. Evaluation metrics
5. Lab: Classifcation Methods

Motivation

The linear regression model assumes that the response variable Y is quantitative.

- Predicting a qualitative response for an observation can be referred to as **classifying** that observation, since it involves assigning the observation to a category, or class.
- On the other hand, often the methods used for classification first **predict the probability** that the observation belongs to each of the categories of a qualitative variablen.

Why not regression?

1. A regression method cannot accommodate a qualitative response with more than two classes;
2. A regression method will not provide meaningful estimates of $Pr(Y|X)$, even with just two classes.

Motivation

The linear regression model assumes that the response variable Y is quantitative.

- Predicting a qualitative response for an observation can be referred to as **classifying** that observation, since it involves assigning the observation to a category, or class.
- On the other hand, often the methods used for classification first **predict the probability** that the observation belongs to each of the categories of a qualitative variablen.

Why not regression?

1. A regression method cannot accommodate a qualitative response with more than two classes;
2. A regression method will not provide meaningful estimates of $Pr(Y|X)$, even with just two classes.

What is Classification?

Definition

Classification is a type of supervised machine learning where the goal is to categorize input data into predefined classes or labels.

Types of Classification Algorithms

1. Linear Classifiers:
   - Logistic Regression
   - Support Vector Machines (SVM)
2. Nearest Neighbor Classifiers:
   - k-Nearest Neighbors (k-NN)
3. Tree-Based Classifiers:
   - Decision Trees
   - Gradient Boosting Machines (e.g., XGBoost)
4. Naive Bayes Classifiers

Outline

1. Classification
2. Logistic Regression
   - The Logistic Model
   - Estimating the Regression Coefficients
   - Multiple Logistic Regression
   - Multinomial Logistic Regression
3. SVM
4. Evaluation metrics
5. Lab: Classifcation Methods

The `Default` dataset (credit risk example)

We study a binary classification problem: default $\in \{\text{Yes}, \text{No}\}$
The goal is to predict whether an individual will default on their credit card payment.

- Default: missing a required credit card payment
- Balance: average monthly *credit card debt* (not bank money)
- Income: annual income of the individual

Key observations

- High balances are strongly associated with default
- Income alone is a weak predictor
- Classes overlap $\Rightarrow$ no perfect separation

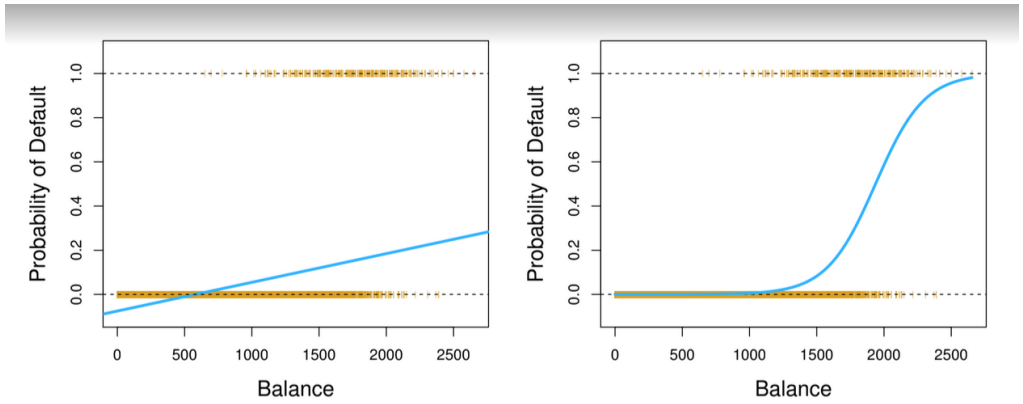
Logistic Regression

Key Note

Rather than modeling this response Y directly, logistic regression models the probability that Y belongs to a particular category.

$$p(X) = \beta_0 + \beta_1(X) \tag{1}$$

Example: For the Default data, we model Pr(default = Yes|balance).



Outline

1 Classification

2 Logistic Regression
- The Logistic Model
- Estimating the Regression Coefficients
- Multiple Logistic Regression
- Multinomial Logistic Regression

3 SVM

4 Evaluation metrics

5 Lab: Classifcation Methods

The Logistic Model

Problem

For balances close to zero, we predict a negative probability of default; if we were to predict for very large balances, we would get values bigger than 1.

To avoid this problem, we must model p(X) using a function that gives outputs between 0 and 1 for all values of X. Many functions meet this description. In logistic regression, we use the logistic function,

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1 + e^{\beta_0+\beta_1 X}} \tag{2}$$

The Logistic Model

- After a bit of manipulation of (2), we find that:

$$\frac{p(X)}{1-p(X)} = e^{\beta_0+\beta_1 X} \tag{3}$$

- The left-hand side is called the odds.
- By taking the logarithm of both sides of (3), we arrive at

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X.$$

- The left-hand side is called the log odds or logit.

Outline

1 Classification

2 Logistic Regression
- The Logistic Model
- Estimating the Regression Coefficients
- Multiple Logistic Regression
- Multinomial Logistic Regression

3 SVM

4 Evaluation metrics

5 Lab: Classifcation Methods

Estimating the Regression Coefficients- The method of maximum likelihood

The basic intuition behind using maximum likelihood

We seek estimates for β_0 and β_1 such that the predicted probability $\hat{P(xi)}$ of default for each individual, using (2), corresponds as closely as possible to the individual's observed default status.

This intuition can be formalized using a mathematical equation called a likelihood function:

$$\ell(\beta_0, \beta_1) = \prod_{i:y_i=1} p(x_i) \prod_{i':y_{i'}=0} (1 - p(x_{i'})).$$

Why Maximum Likelihood for Logistic Regression?

Key idea (high level)

In logistic regression, we model probabilities of a binary outcome, not the outcome itself.

- In linear regression: coefficients are estimated by least squares (closed form).
- In logistic regression: $y_i \in \{0, 1\}$ and $\hat{p}_i$ must be in $[0, 1]$.
- We choose coefficients β that make the **observed labels** most plausible under the model.

$$p_i = \mathbb{P}(Y_i = 1 \mid x_i) = \sigma(\eta_i), \quad \eta_i = \beta_0 + \beta^\top x_i, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

From Individual Probabilities to a Likelihood

- For each observation i:
  - if $y_i = 1$, the model assigns probability p_i
  - if $y_i = 0$, the model assigns probability $1 - p_i$
- The likelihood multiplies contributions across all observations.

Likelihood of the full sample

$$L(\beta) = \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1 - y_i}$$

- Maximizing $L(\beta)$ means: predicted probabilities match observed outcomes as well as possible.

Why Use the Log-Likelihood?

- Products of many probabilities can be numerically unstable.
- Taking logs converts products into sums (easier to optimize).
- Maximizing $L(\beta)$ is equivalent to maximizing $\ell(\beta) = \log L(\beta)$.

Log-likelihood

$$\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right]$$

- Intuition: each observation contributes a reward/penalty depending on how well it is predicted.

How Are the Coefficients Found in Practice?

Important practical point

Unlike linear regression, logistic regression does **not** have a simple closed-form solution for $\hat{\beta}$.

- We solve:

$$\hat{\beta} = \arg\max_{\beta} \ \ell(\beta)$$

- The log-likelihood is smooth and (for standard logistic regression) concave $\Rightarrow$ a unique optimum.
- Numerical algorithms used in software:
  - Gradient ascent / quasi-Newton (e.g., BFGS)
  - Newton–Raphson / IRLS (Iteratively Reweighted Least Squares)

In practice: R / Python / MATLAB compute $\hat{\beta}$ automatically.

Interpreting Logistic Regression Coefficients

- Logistic regression models the **log-odds**:

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta^\top x$$

- Interpretation of β_j:
  - a one-unit increase in x_j changes the log-odds by β_j
  - holding other predictors constant
- Exponentiating gives an **odds ratio**: $\exp(\beta_j) =$ multiplicative change in the odds for a 1-unit increase in x_j

Quick intuition

$\beta_j > 0$ increases odds (and usually probability); $\beta_j < 0$ decreases odds.

Summary

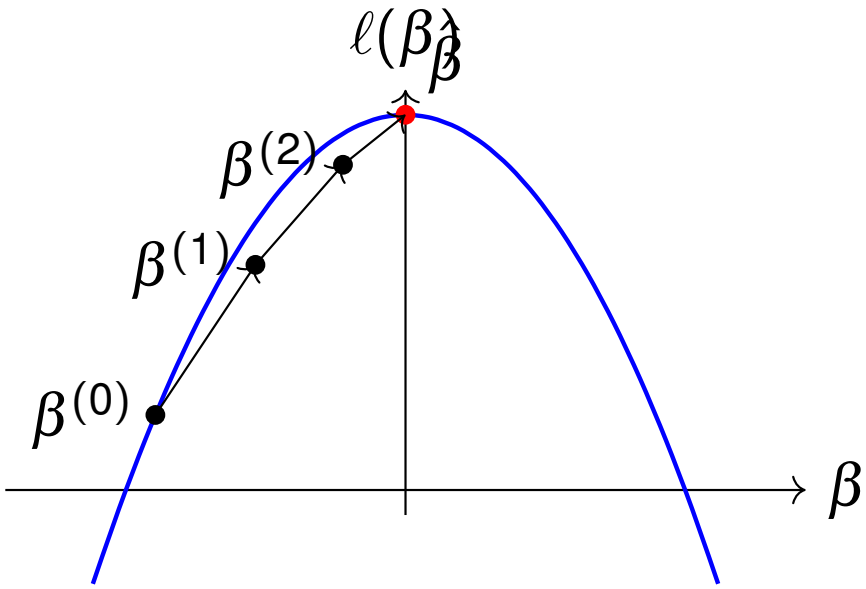
Logistic regression: estimation and meaning

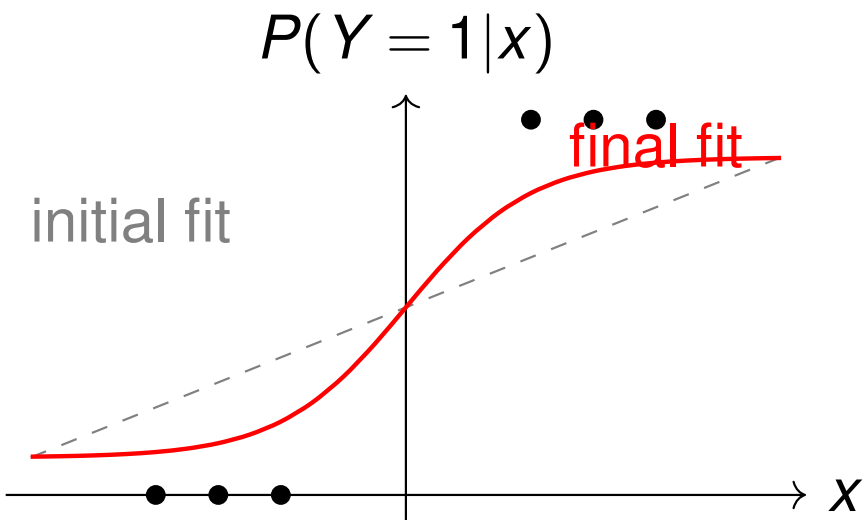
- We model probabilities: $p_i = \mathbb{P}(Y_i = 1 \mid x_i)$.
- Coefficients are estimated by **maximum likelihood**.
- We maximize the log-likelihood:

$$\ell(\beta) = \sum_{i=1}^{n} \Big[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \Big].$$

- Optimization is numerical (IRLS / Newton methods).
- Coefficients are interpreted via log-odds and odds ratios.

Visual Intuition





Example Estimating the Regression Coefficients

Table 1 shows the coefficient estimates and related information that result from fitting a logistic regression model on the Default data in order to predict the probability of default=Yes using balance.

	Coefficient	Std. error	z-statistic	p-value
`Intercept`	-10.6513	0.3612	-29.5	<0.0001
`balance`	0.0055	0.0002	24.9	<0.0001

For the Default data, estimated coefficients of the logistic regression model that predicts the probability of default using balance. A one-unit increase in balance is associated with an increase in the log odds of default by 0.0055 units.

Outline

1 Classification

2 Logistic Regression
- The Logistic Model
- Estimating the Regression Coefficients
- Multiple Logistic Regression
- Multinomial Logistic Regression

3 SVM

4 Evaluation metrics

5 Lab: Classifcation Methods

Multiple Logistic Regression

By analogy with the extension from simple to MLR , we can generalize (4) as follows, where X=($X_1, \ldots,_p$) are p predictors.

$$\log \frac{\mathrm{p(X)}}{1-\mathrm{p(X)}} = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

Table 2 shows the coefficient estimates that uses balance, income, and student status to predict probability of default.

	Coefficient	Std. error	z-statistic	p-value
`Intercept`	−10.8690	0.4923	−22.08	<0.0001
`balance`	0.0057	0.0002	24.74	<0.0001
`income`	0.0030	0.0082	0.37	0.7115
`student[Yes]`	−0.6468	0.2362	−2.74	0.0062

Student status is encoded as a dummy variable student[Yes], with a value of 1 for a student.

Interpreting Coefficients in Multiple Logistic Regression

- The model describes changes in **log-odds**

$$\log\left(\frac{p(X)}{1-p(X)}\right)$$

Interpretationof a single coefficient β_j

A one-unit increase in X_j changes the log-odds of $Y = 1$ by β_j, **holding all other predictors fixed**.

- This is different from simple logistic regression
- Effects may change sign or significance when new variables are added

Outline

1 Classification

2 Logistic Regression
- The Logistic Model
- Estimating the Regression Coefficients
- Multiple Logistic Regression
- Multinomial Logistic Regression

3 SVM

4 Evaluation metrics

5 Lab: Classifcation Methods

Beyond Binary Logistic Regression

- Logistic regression handles $Y \in \{0, 1\}$
- Many problems involve more than two classes:
  - disease type
  - risk level
  - object category
- We need a model for $Y \in \{1, \ldots, K\}$

Solution

Multinomial (multiclass) logistic regression

Reference Class Idea

- One class is chosen as a baseline (reference)
- Without loss of generality, choose class K
- All other classes are compared to this reference

Why do we need a reference?

- To avoid redundancy
- To ensure model identifiability

Multinomial Logistic Regression Model

For $k = 1, \ldots, K-1$:

$$\Pr(Y = k \mid X = x) = \frac{e^{\beta_{k0} + \beta_k^\top x}}{1 + \sum_{\ell=1}^{K-1} e^{\beta_{\ell 0} + \beta_\ell^\top x}}$$

For the reference class:

$$\Pr(Y = K \mid X = x) = \frac{1}{1 + \sum_{\ell=1}^{K-1} e^{\beta_{\ell 0} + \beta_\ell^\top x}}$$

Key property

Probabilities are positive and sum to 1

Interpretation via Log-Odds

For $k = 1, \ldots, K-1$:

$$\log\left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)}\right) = \beta_{k0} + \beta_k^\top x$$

Interpretation

- Each class has its own regression equation
- Coefficients compare class k to the reference class
- Effects are interpreted **holding other predictors fixed**

Example

- *Consider an emergency room setting with three possible medical conditions: stroke, drug overdose, and epileptic seizure.*
- *Multinomial logistic regression requires choosing one category as a baseline (reference) class. For example, we may choose stroke or drug overdose as the baseline.*
- *Changing the baseline affects the numerical values of the coefficients, but **does not affect**:*
  - *the predicted probabilities,*
  - *the log-odds between any pair of classes,*
  - *or the overall model fit.*
- *Therefore, coefficient interpretation must always be made relative to the chosen baseline.*

Multinomial Logistic Regression: Softmax Coding

- In softmax coding, **no reference class is chosen**.
- All K classes are treated **symmetrically**.
- Each class k is assigned a linear score:

$$\eta_k(x) = \beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p$$

Softmax probabilities

$$\Pr(Y = k \mid X = x) = \frac{e^{\eta_k(x)}}{\sum_{\ell=1}^{K} e^{\eta_\ell(x)}}$$

- The denominator ensures probabilities are positive and sum to 1.

Interpretation of Softmax Coefficients

Log-odds between two classes

$$\log\left(\frac{\Pr(Y=k \mid X=x)}{\Pr(Y=k' \mid X=x)}\right) = (\beta_{k0} - \beta_{k'0}) + (\beta_k - \beta_{k'})^\top x$$

- Interpretation is always relative between two classes.
- Individual coefficients β_k have no standalone meaning.
- Only differences between coefficients matter.

Key message

Softmax coding and baseline coding give the same predictions; they are just two equivalent parameterizations.

Outline

1. Classification
2. Logistic Regression
3. SVM
   - The SVC model
   - Kernel Trick
   - Implemention in Python
   - Multi-class SVM
4. Evaluation metrics
5. Lab: Classifcation Methods

Outline

1 Classification

2 Logistic Regression

3 SVM
- The SVC model
- Kernel Trick
- Implemention in Python
- Multi-class SVM

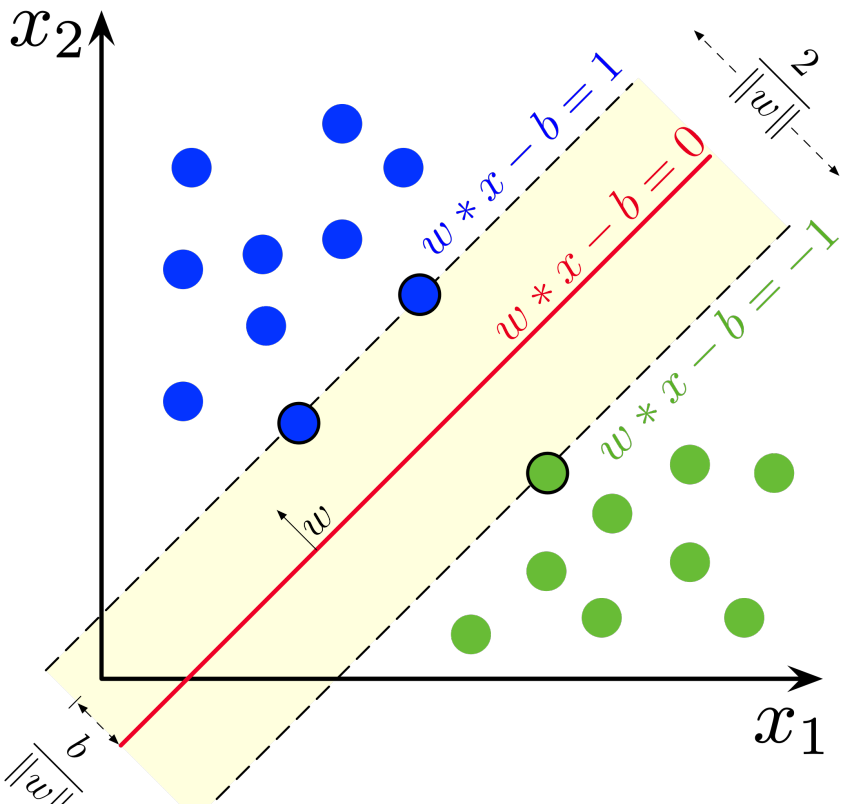
4 Evaluation metrics

5 Lab: Classifcation Methods

Support Vector Machine (SVM)

Definition

A Support Vector Machine (SVM) is a supervised learning algorithm that finds an optimal hyperplane separating two classes.



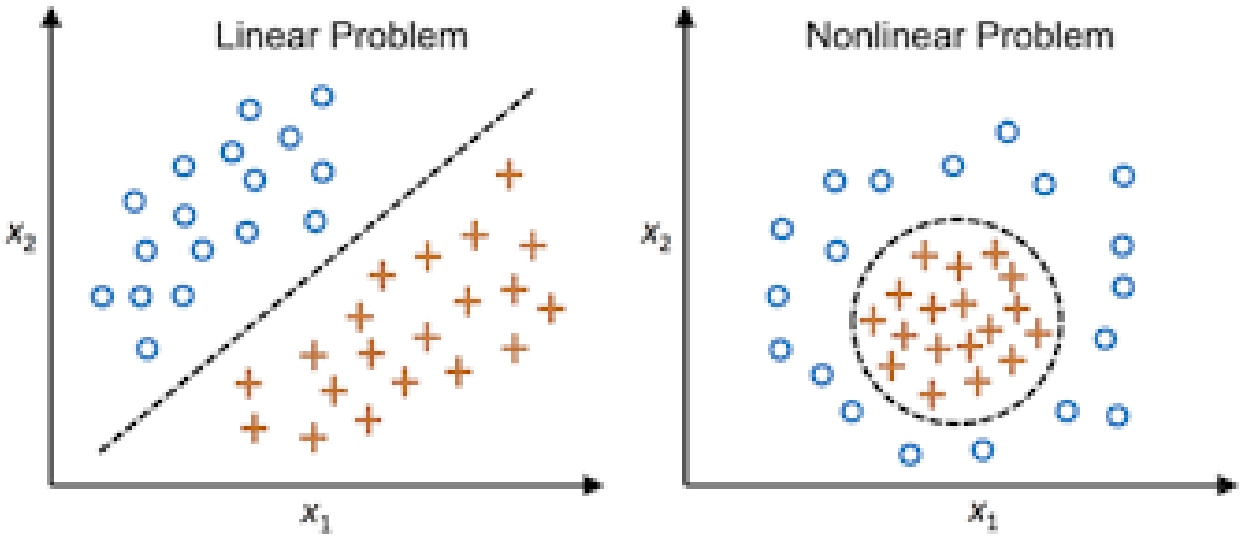
Linearly vs Nonlinearly Separable Data

Linearly separable

- A straight line separates classes
- Simple hyperplane works

Nonlinearly separable

- No straight line can separate classes
- Need feature transformation

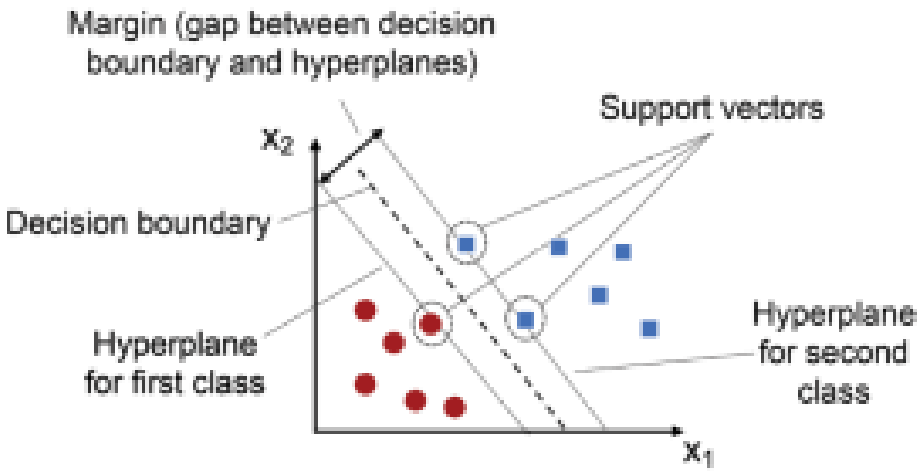


Maximum Margin Principle

Key Idea

Among all separating hyperplanes, SVM chooses the one that maximizes the margin Margin $= \frac{2}{\|\mathbf{w}\|}$.

- Margin = distance to closest points
- Closest points = **Support Vectors**



Hard-Margin SVM (Linearly Separable Data)

Optimization problem

$$\min_{\mathbf{w},b} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

subject to: $y_i(\mathbf{w}^T x_i + b) \geq 1$

Interpretation

- Data are perfectly separable
- No misclassification allowed
- Maximizing margin $\iff$ minimizing $\|\mathbf{w}\|^2$
- Margin width = $\frac{2}{\|\mathbf{w}\|}$

Limitation

Real datasets are rarely perfectly separable.

Why Do We Need Soft Margin?

Problem in practice

- Noise
- Outliers
- Overlapping classes

Idea

Allow violations of the margin constraint using slack variables ξ_i:

$$y_i(\mathbf{w}^T x_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

- $\xi_i = 0$ correctly classified outside margin
- $0 < \xi_i < 1$ inside margin
- $\xi_i > 1$ misclassified

Soft-Margin SVM (Primal Form)

$$\min_{\mathbf{w},b,\xi} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

subject to:

$$y_i(\mathbf{w}^T x_i + b) \geq 1 - \xi_i \quad , \quad \xi_i \geq 0$$

Interpretation

- $\frac{1}{2}\|\mathbf{w}\|^2 \Rightarrow$ maximize margin
- $C\sum\xi_i \Rightarrow$ penalize violations
- C controls trade-off:
  - Small C wider margin, more tolerance
  - Large C stricter classification

Key Intuitions

We want to allow some violations of the margin using slack variables ξ_i.

- $\|w\|^2$ controls the margin size
- ξ_i measures how much point i violates the margin
- C controls how strongly violations are penalized

The constraint

$$y_i(w^\top x_i + b) \geq 1 - \xi_i$$

can be rewritten as

$$\xi_i \geq 1 - y_i(w^\top x_i + b)$$

Therefore ξ_i must satisfy

$$\xi_i \geq \max(0,\ 1 - y_i(w^\top x_i + b))$$

From Slack Variables to Hinge Loss

For fixed (w, b), the optimizer will choose the smallest feasible value of ξ_i. Since we minimize

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

the optimal slack value is

$$\xi_i^\star = \max(0,\ 1 - y_i(w^\top x_i + b))$$

Substituting this into the objective gives

$$\min_{w,b}\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,\ 1 - y_i(w^\top x_i + b))$$

Interpretation

Let

$$m_i = y_i f(x_i)$$

be the signed margin.

Situation	Margin m_i	Loss
Correct and outside margin	$m_i \geq 1$	0
Inside margin	$0 < m_i < 1$	$1 - m_i$
Misclassified	$m_i \leq 0$	> 1

Key intuition

Hinge loss penalizes margin violations, not only misclassifications.

SVM as Regularized Risk Minimization

The SVM objective becomes

$$\min_{w,b} \underbrace{\sum_{i=1}^{n} \ell_{\text{hinge}}(y_i, f(x_i))}_{\text{regularization}} + \underbrace{\lambda \|w\|^2}_{\text{data fitting}}$$

General ML framework

$$\text{Loss} + \lambda \times \text{Penalty}$$

Interpretation

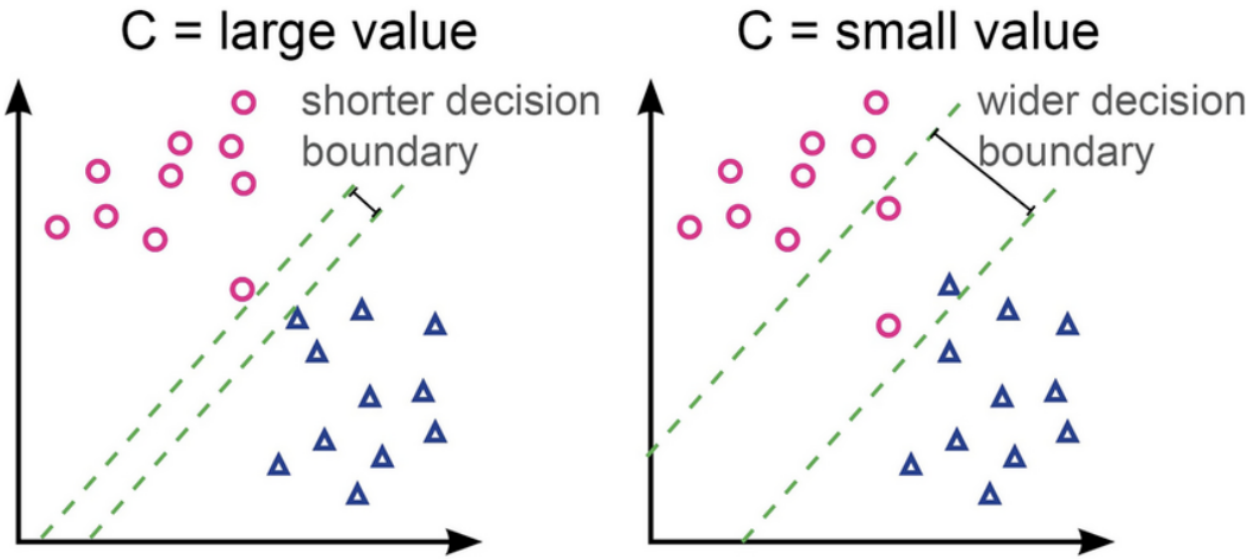
- Loss: penalizes margin violations
- Regularization: encourages a large margin
- Trade-off controlled by C (or λ)

Role of Penality Parameter C

- Large $C \Rightarrow$ small margin low tolerance to errors
- Small $C \Rightarrow$ large margin more tolerance to errors

Bias-Variance Tradeoff

- Large $C \rightarrow$ risk of overfitting
- Small $C \rightarrow$ risk of underfitting



Outline

1 Classification

2 Logistic Regression

3 SVM
- The SVC model
- Kernel Trick
- Implemention in Python
- Multi-class SVM

4 Evaluation metrics

5 Lab: Classifcation Methods

From Primal to Dual Formulation

Soft-margin SVM (primal):

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

subject to: $y_i(w^T x_i + b) \geq 1 - \xi_i$

Dual problem

$\max_\alpha \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$; $0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0$

Key observation

Data appear only through inner products $\langle x_i, x_j \rangle$

Mathematical Definition of a Kernel

Definition

A function

$$K : \mathscr{X} \times \mathscr{X} \to \mathbb{R}$$

is a **kernel** if there exists a feature map $\phi : \mathscr{X} \to \mathscr{H}$ such that:

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathscr{H}}$$

Positive Definite Condition

For any finite set $\{x_1, \ldots, x_n\}$, the kernel matrix $K_{ij} = K(x_i, x_j)$ must be symmetric and positive semi-definite:

$$\sum_{i,j} c_i c_j K(x_i, x_j) \geq 0$$

Mercer's Theorem (Intuition)

Mercer's Theorem

If K is continuous, symmetric and positive definite, then there exists a (possibly infinite-dimensional) feature map ϕ such that:

$$K(x, x') = \langle \phi(x), \phi(x') \rangle$$

- We never need to compute $\phi(x)$ explicitly
- Computations happen in input space
- Equivalent to working in a Reproducing Kernel Hilbert Space (RKHS)

Common Kernel Functions

1. Linear Kernel

$$K(x, x') = x^T x'$$

Equivalent to classical linear SVM.

2. Polynomial Kernel

$$K(x, x') = (x^T x' + c)^d$$

- d: degree
- c: offset

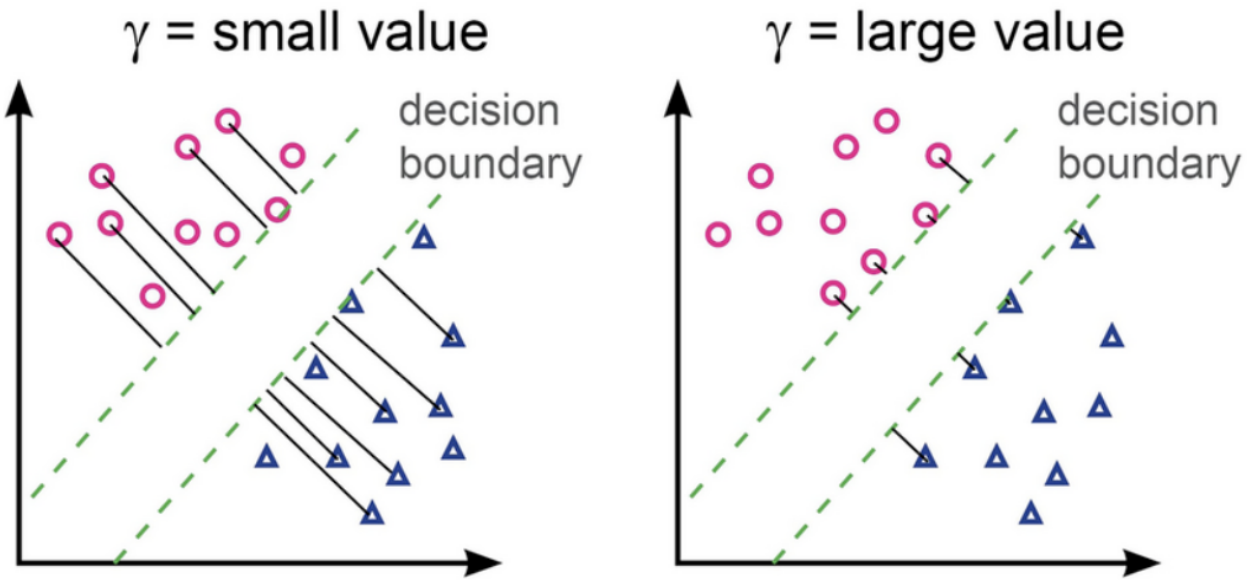
3. Gaussian (RBF) Kernel

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Understing the RBF Kernel

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right)$$

- Small γ: smoother boundary
- Large γ: complex boundary



Other Valid Kernels

Sigmoid Kernel

$$K(x, x') = \tanh(\kappa x^T x' + c)$$

Related to neural networks.

Laplacian Kernel

$$K(x, x') = \exp(-\gamma \|x - x'\|_1)$$

String / Graph Kernels

Kernels can be defined on:

- Text sequences
- Graphs
- Images
- Structured objects

Designing Custom Kernels

Kernel Engineering

You can define your own kernel as long as it is:

- Symmetric
- Positive semi-definite

Closure Properties

If K_1 and K_2 are valid kernels, then:

- $K = K_1 + K_2$ is valid
- $K = K_1 \cdot K_2$ is valid
- $K = aK_1$ (for $a > 0$) is valid

Outline

1 Classification

2 Logistic Regression

3 SVM
- The SVC model
- Kernel Trick
- Implemention in Python
- Multi-class SVM

4 Evaluation metrics

5 Lab: Classifcation Methods

Implemention in Python

Main hyperparameters

- `C`: regularization parameter $\rightarrow$ Controls margin vs classification errors Small C = wider margin (more regularization) Large C = fewer errors (less regularization)
- `kernel`: transformation of feature space
  - `"linear"` â linear decision boundary
  - `"poly"` â polynomial boundary
  - `"rbf"` â non-linear Gaussian boundary
- `gamma` (for rbf/poly kernels): Controls how far the influence of a point reaches
- `degree`: degree of polynomial (only if kernel="poly")

SVM Implementation in Python (scikit-learn)

Import

```
from sklearn.svm import SVC
```

Model definition

```
model = SVC(
    C=1.0,
    kernel="rbf",
    gamma="scale",
    degree=3,
    probability=True
)
```

Training and prediction

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Using a Custom Kernel in scikit-learn

```
def custom_kernel(X, Y):
    return (X @ Y.T + 1)**2

from sklearn.svm import SVC

model = SVC(kernel=custom_kernel)
model.fit(X_train, y_train)
```

Important

Kernel must produce a positive semi-definite matrix.

Outline

1 Classification

2 Logistic Regression

3 SVM
- The SVC model
- Kernel Trick
- Implemention in Python
- Multi-class SVM

4 Evaluation metrics

5 Lab: Classifcation Methods

Multi-class SVM

Binary nature of SVM

The standard SVM decision function is binary:

$$f(x) = \mathbf{w}^\top \phi(x) + b \quad \Rightarrow \quad \hat{y} = \mathrm{sign}(f(x)) \in \{-1, +1\}.$$

Multi-class setting

For K classes, we need a rule to combine several binary SVMs into:

$$\hat{y} \in \{1, \ldots, K\}.$$

- Two standard decompositions: **One-vs-Rest (OvR)** and **One-vs-One (OvO)**
- Both are widely used; trade-off is mainly computational and interface / outputs

One-vs-Rest (OvR): K Classifiers

Construction

Train K binary classifiers. For class k:

$$f_k(x) = \mathbf{w}_k^\top \phi(x) + b_k,$$

where positives are class k and negatives are all other classes.

Prediction:

$$\hat{y} = \arg\max_{k \in \{1,\dots,K\}} f_k(x).$$

Pros / Cons

- + Only K models
- + Natural decision scores of shape (n_{samples}, K)
- – Each classifier faces a potentially imbalanced problem (one class vs all)

One-vs-One (OvO): $\frac{K(K-1)}{2}$ Classifiers

Construction

Train one classifier for each pair of classes (k, ℓ):

$$f_{k\ell}(x) = \mathbf{w}_{k\ell}^{\top}\phi(x) + b_{k\ell}.$$

Number of classifiers:

$$\frac{K(K-1)}{2}.$$

Prediction (concept)

Each classifier votes for one of its two classes; final class is obtained by:

$$\hat{y} = \arg\max_{k} \text{votes}(k)$$

(or by aggregating margins / decision values depending on implementation).

What scikit-learn Does (SVC vs LinearSVC)

Key implementation facts

- `SVC` (kernel SVM) trains internally using **OvO**
- Output interface can be formatted as OvR-shaped using: `decision_function_shape="ovr"`
- `LinearSVC` uses OvR directly (trains K models)

```
from sklearn import svm
X = [[0],[1],[2],[3]]
y = [0,1,2,3]
clf = svm.SVC(decision_function_shape="ovo")
clf.fit(X,y)
clf.decision_function_shape = "ovr"
lin = svm.LinearSVC()
lin.fit(X,y)
```

Choosing OvO vs OvR in Practice

Rule of thumb

- If K is small (e.g., $K \leq 10$): OvO is often fine and can be strong.
- If K is large: OvR is typically cheaper (only K models).

Kernel vs Linear

- Kernel SVM (`SVC`): powerful but scaling can be costly with large n
- Linear SVM (`LinearSVC` / `SGDClassifier` with hinge loss): good for high-dimensional features and large datasets

Important practical note

Always scale features (e.g., `StandardScaler`) before SVM.

Outline

1. Classification
2. Logistic Regression
3. SVM
4. Evaluation metrics
5. Lab: Classifcation Methods

Confusion Matrix: the foundation

Binary classification outcomes

	Predicted 0	**Predicted 1**
Actual 0	TN	FP
Actual 1	FN	TP

- TP: true positives (correctly detected positives)
- FP: false positives (false alarms)
- FN: false negatives (missed positives)
- TN: true negatives

Accuracy

Definition

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Measures overall correctness
- Easy to interpret

Limitation

Accuracy can be misleading when classes are imbalanced.

`sklearn.metrics.accuracy_score`

Precision and Recall

Definitions

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

- **Precision**: How many predicted positives are correct?
- **Recall**: How many actual positives are detected?

Interpretation

- High precision $\Rightarrow$ few false alarms
- High recall $\Rightarrow$ few missed detections

`precision_score, recall_score`

F1-score

Definition

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Harmonic mean of precision and recall
- Penalizes extreme imbalance between the two

When to use F1-score

When both false positives and false negatives matter.

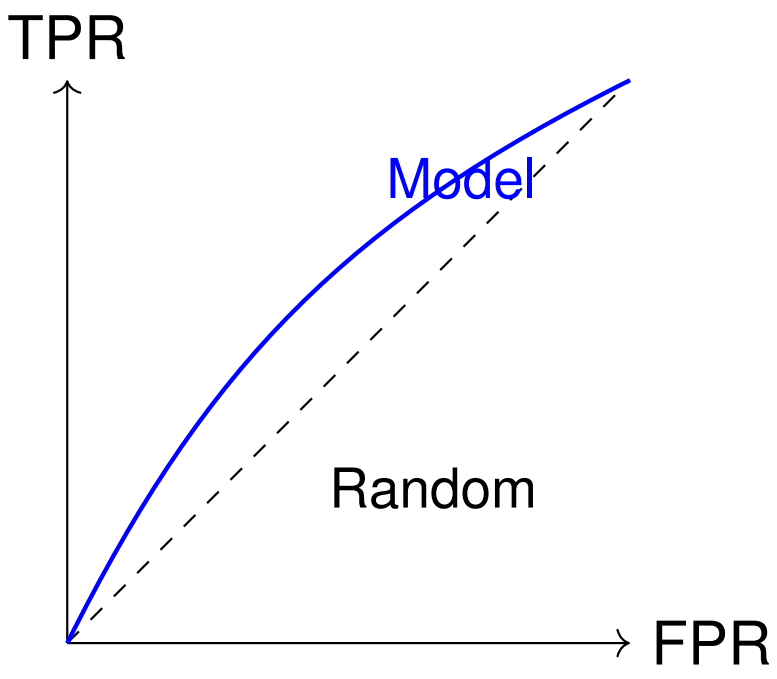
`sklearn.metrics.f1_score`

ROC Curve

Key idea

The ROC curve evaluates performance across **all decision thresholds**.

- TPR = Recall = $\frac{TP}{TP+FN}$
- FPR = $\frac{FP}{FP+TN}$
- Each point = one threshold



AUC: Area Under the Curve

Definition

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR})$$

- Threshold-independent metric
- Probability that a random positive is ranked above a random negative

Interpretation

- AUC = 0.5 (random guessing)
- AUC = 1.0 (perfect classifier)

`sklearn.metrics.roc_auc_score`

Outline

1. Classification
2. Logistic Regression
3. SVM
4. Evaluation metrics
5. Lab: Classifcation Methods

Lab 1: Social_Network_Ads (Logistic Regression)

Objective

Build a logistic regression classifier to predict whether a user purchases a product (`Purchased` $\in \{0, 1\}$) using two predictors: `Age` and `EstimatedSalary`.

The dataset contains user profiles from a social network advertisement campaign. Each row corresponds to one user.

Instructions

- Open a Python notebook (Jupyter or Google Colab).
- Create one section per step of the lab (Step 1 till 5).
- For each step: include your code, the requested plots/tables, and short answers to the questions.

Deadline: March 24

Step 1: Load & Inspect the Dataset

- Load the CSV file and inspect the table structure
- Identify predictors (`Age`, `EstimatedSalary`) and target (`Purchased`)
- Check missing values and class balance

Questions

- How many observations and variables?
- Is the target balanced?
- Any missing values?

Python hints

`pd.read_csv`, `df.head()`, `df.info()`, `value_counts()`, `isna().sum()`

Step 1: Load & Inspect the Dataset

- Load the CSV file and inspect the table structure
- Identify predictors (`Age`, `EstimatedSalary`) and target (`Purchased`)
- Check missing values and class balance

Questions

- How many observations and variables?
- Is the target balanced?
- Any missing values?

Python hints

`pd.read_csv`, `df.head()`, `df.info()`, `value_counts()`, `isna().sum()`

Step 2: Visualize the Classes (Separability?)

- Create a scatter plot in feature space: `Age` (x-axis) vs `EstimatedSalary` (y-axis)
- Color points by `Purchased` (0/1)

Questions

- Do the classes appear (approximately) linearly separable?
- What would a linear decision boundary look like here?

Python hints

- `import matplotlib.pyplot as plt`
- `plt.scatter(x, y, c=labels)`
- Color options such as `cmap="bwr"` and transparency `alpha=0.7`
- Axis labels with `plt.xlabel()` and `plt.ylabel()`

Step 3: Fit Logistic Regression (Train/Test)

- Split data into training and test sets
- Fit logistic regression on (`Age`, `EstimatedSalary`)
- Extract coefficients and interpret signs

Questions

- What are the signs of the coefficients?
- Which predictor increases the odds of purchase?

Python hints

- Split the data: `train_test_split` from `sklearn.model_selection`
- Fit the model: `LogisticRegression` from `sklearn.linear_model`
- Access coefficients: `clf.coef_` and `clf.intercept_`

Step 4: Predictions and Model Evaluation

- Use the trained model to predict:
  - class labels (0/1)
  - predicted probabilities
- Evaluate how well the model performs on the test set

Questions

- What is the difference between predicted probabilities and predicted classes?
- How accurate is the model on the test set?
- Is accuracy alone sufficient to evaluate performance?

Python hints

- `clf.predict(X_test)`
- `clf.predict_proba(X_test)`
- `accuracy_score` from `sklearn.metrics`

Step 5: Effect of the Classification Threshold

Logistic regression outputs probabilities in $[0, 1]$. A threshold τ is used to convert probabilities into class labels; We usually fix $\tau = 0.5$, but this is not always optimal

- Recompute confusion matrices for different values of τ

Questions

- What happens if the threshold is lowered (e.g. $\tau = 0.3$)?
- What happens if the threshold is increased (e.g. $\tau = 0.7$)?
- Which types of errors are affected (false positives vs false negatives)?

Python hints

- `clf.predict_proba(X_test)[:,1]`
- Convert to labels: `(proba >= tau).astype(int)`

Step 6: Regularization

Logistic regression estimates coefficients by maximizing the likelihood. However, this can lead to overfitting when:

- predictors are strongly correlated,
- the dataset is small, or we include many features.

Regularized logistic regression maximizes a penalized log-likelihood:

$$\hat{\beta} = \arg\max_{\beta} \Big(\ell(\beta) - \lambda \, \mathscr{P}(\beta) \Big)$$

- $\ell(\beta)$: log-likelihood (fit to the data)
- $\mathscr{P}(\beta)$: penalty on model complexity

Penalty terms

- L2 (Ridge): $\mathscr{P}_{L2}(\beta) = \sum_j \beta_j^2$
- L1 (Lasso): $\mathscr{P}_{L1}(\beta) = \sum_j |\beta_j|$

Step 6: Regularization

- L2 regularization (Ridge)
  - Penalizes the square of coefficients
  - Shrinks coefficients smoothly toward zero
  - Usually keeps all predictors
- L1 regularization (Lasso)
  - Penalizes the absolute value of coefficients
  - Can set some coefficients exactly to zero
  - Performs implicit feature selection

Intuition

- L2:Use all variables, but more cautiously.
- L1: Use only the most important variables.

Questions

- Which regularization would you choose if interpretability is important? Which one would you expect to be more stable?

Step 6: Regularization

In `scikit-learn`, regularization is controlled by the parameter `C`.

- **Large** $C \Rightarrow$ weak regularization (model fits data closely)
- **Small** $C \Rightarrow$ strong regularization (coefficients shrink)

Questions

- Try several values of `C` (e.g. 10, 1, 0.1, 0.01).
- How do the coefficients change as C decreases?
- How does test accuracy change?

Python hints

- `LogisticRegression(penalty="l2", C=C)`
- `LogisticRegression(penalty="l1", solver="liblinear", C=C)`